

GeoRoute

Pan-India Offline Map & Routing Webapp

Detailed Technical Documentation

Architecture, data model, ingest, snap, routing algorithms,
basemap tiles, REST APIs, frontend, and offline portable packaging.

Stack: Angular 17 + OpenLayers | Spring Boot 3.2 | JDK 17
PostgreSQL + PostGIS | GeoTools 29.2

This document supersedes the shorter overview PDF with operational detail
of how the system behaves at country / city scale.

Table of contents

- 1. Project overview and design goals
- 2. Technology stack
- 3. Repository layout
- 4. Database schema (PostGIS)
- 5. Backend components (detailed)
- 6. Algorithms in depth (ingest, snap, route, tiles)
- 7. Flowcharts and architecture diagrams
- 8. Frontend components and UX flow
- 9. REST API contracts
- 10. Configuration reference
- 11. Dev vs portable offline package
- 12. Known pitfalls and performance notes
- 13. Summary

1. Project overview and design goals

GeoRoute is an offline-capable geospatial routing application. It loads a local road Shapefile into PostGIS, optionally renders a local GeoTIFF as XYZ map tiles, snaps map clicks to the nearest road, and computes one or more routes using hand-written A* or Dijkstra search (no commercial routing engines, no cloud map APIs).

Design goals

- Runtime offline: zero outbound HTTP for maps, routing, fonts, or tiles.
- Scale: handle multi-million edge OSM extracts (ingest streaming; query by bbox).
- Usability: OpenLayers map, algorithm toggle, K alternates, snap pins, toasts.
- Portability: prebuild a fat JAR that serves UI + API on one port (no npm on target).

Runtime topology (development)

```
Browser -> Angular ng serve (:4200)
          | proxy /api/*
          v
Spring Boot (:8080)
  |-- JDBC -----> PostgreSQL + PostGIS
  |-- GeoTools -> roads Shapefile (startup ingest)
  +-- GeoTools -> GeoTIFF (tile render)
```

Runtime topology (portable offline-bundle)

```
Browser -> http://localhost:8080/
          | static Angular from JAR classpath:/static/
          | /api/* Spring controllers
          v
java -jar georoute-backend-1.0.0.jar
    +-- same PostGIS + local data/ files
```

2. Technology stack

Layer	Technology	Role
Frontend	Angular 17, OpenLayers, TS	Map UI, pins, routes, sidebar
Backend	Spring Boot 3.2, Java 17	REST, ingest, snap, route, tiles
GIS libs	GeoTools 29.2, JTS	Shapefile + GeoTIFF processing
Database	PostgreSQL + PostGIS	Spatial graph + GIST indexes
Build	Maven, npm / Angular CLI	Dev builds; offline-bundle packing

Layer	Technology	Role
Package	Spring Boot fat JAR + SPA	Air-gapped single-process deploy

3. Repository layout

Maps_shrushru_DRDO/	
db/init.sql	PostGIS schema
data/	GIS assets (gitignored): SHP, GeoTIFF
georoute-backend/	Spring Boot app
controller/	REST
service/	DataLoader, Snap, Routing, GeoTIFF
routing/	RoadGraph (A*/Dijkstra/alternates)
repository/	JDBC helpers
config/	CORS, SpaWebConfig, PostGIS
georoute-frontend/	Angular app
components/	map, route-panel, toast
services/	snap, route, map-state
scripts/	
package-offline.ps1	Build offline-bundle/
rebuild_road_basemap.py	RGB road basemap from PostGIS
generate_tech_doc.py	This PDF
package-offline.bat	Wrapper for packaging
offline-bundle/	Generated portable folder (not committed)
setup.bat / setup.sh	One-time dependency install

4. Database schema (PostGIS)

Schema is owned by db/init.sql. Hibernate ddl-auto=none. Extensions: postgis, postgis_topology.

4.1 Tables

Table	Purpose	Key columns / indexes
system_config	Ingest flags	key PK, value; shp_loaded=true
road_nodes	Graph vertices	id, coord_key UNIQUE, geom Point4326, GIST(geom)
road_edges	Road segments	source/target FKs, LineString, length_m, speed_kmh, oneway, cost; GIST(...

4.2 Cost model

Edge cost is travel time in seconds: $\text{length_m} / (\text{speed_kmh} / 3.6)$. Speeds come from OSM fclass during ingest (motorway ~110 km/h down to tracks/paths ~20). Two-way roads are stored once; RoutingService expands them to bidirectional edges in memory. oneway=true keeps the forward direction only.

5. Backend components (detailed)

5.1 Controllers

Class	Endpoint	Responsibility
SnapController	POST /api/snap	Snap lat/lng to nearest road node
RouteController	POST /api/route	Re-snap endpoints; K routes; GeoJSON
TileController	GET /api/tiles/{z}/{x}/{y}.png	256x256 basemap PNG; no-store
HealthController	GET /api/health	PostGIS, ready flags, bounds, counts

5.2 DataLoaderService

On ApplicationReadyEvent, starts daemon thread georoute-data-loader so Tomcat accepts HTTP while ingest runs. Streams Shapefile features with GeoTools (does not load the entire network into heap). Each LineString is split into consecutive vertex-to-vertex segments so intersections become shared nodes.

- force-reload=true: TRUNCATE nodes/edges, clear shp_loaded, re-ingest once.
- If shp_loaded already true: skip ingest and mark DataLoadStatus ready.
- NodeIdResolver: LRU cache (shapefile.node-cache.size, default 300k) + upsert by coord_key.
- EdgeBatcher: JDBC batch inserts (chunks ~2000).
- Post-ingest: ANALYZE tables; set shp_loaded; DataLoadStatus.ready=true.

5.3 RoadSnapService

Snaps a WGS84 click to the nearest road within 1 km. A naive global KNN (ORDER BY geom <-> point) on 20M+ edges hits the 30s JDBC timeout. Strategy:

- Prefilter: e.geom && ST_Expand(point, 0.015 deg) (~1.5 km) using GIST.
- Among candidates: ORDER BY geom <-> point LIMIT 1.
- Snapped pin = ST_ClosestPoint; routing node = nearer of edge endpoints.
- If distance > 1000 m or no hit: NoRoadNearbyException (404).
- While ingest incomplete: DataNotReadyException (503).

5.4 RoutingService

Does not walk the national graph via per-neighbor JDBC (too slow). Loads a corridor subgraph around source and destination, builds an in-memory RoadGraph, searches locally.

- Straight-line distance ≥ 12 km => longTrip (highway hierarchy).
- Short pad: $\max(0.03 \text{ deg}, \text{span} \cdot 0.25 + 0.015)$. Long pad: slimmer corridor caps.
- Hierarchy: mid-corridor major types; all classes within ~1.2 km of endpoints.
- Edge SQL does NOT join road_nodes (latency); nodes loaded in chunked IN lists.
- PreparedStatement query timeout 90s for subgraph (global JDBC timeout remains 30s).
- Cap: LIMIT 250000 undirected edges.
- Retry: hierarchy miss -> all classes; short miss -> wider pad.

5.5 RoadGraph

In-memory directed adjacency. A* and Dijkstra share best-first search; difference is heuristic $h(n)$: Haversine/maxSpeed (A*) vs 0 (Dijkstra). With an admissible heuristic, both return the same optimal path; A* expands fewer nodes when geometry is good.

Alternate routes (not Yen)

Classic Yen k-shortest often only detours one OSM segment on dense graphs. GeoRoute uses iterative edge-penalty diversification:

- Find shortest path.
- Multiply cost of used edges (and reverse) by 2.5.
- Re-search; reject duplicate signatures and Jaccard overlap > 0.55.
- Repeat until K paths or maxAttempts (~ $\max(k \cdot 8, 8)$). Sort by true cost.
- Labels: index 0 'Fastest Route'; else 'Alternate i'.

5.6 GeoTiffTileService

XYZ (Web Mercator) indices -> WGS84 lon/lat box -> bilinear resample of coverage to 256x256 -> PNG. Disk cache under tiles.cache.dir. Missing GeoTIFF or outside footprint => transparent PNG so the UI still works.

- RGB multi-band: drawRenderedImage.
- Single-band/float: grayscale paint; near-constant fills stay translucent (avoids yellow map bug).
- Recommended basemap: data/roads_basemap.tif (3-band RGB from rebuild_road_basemap.py).

5.7 SpaWebConfig + CorsConfig

- SpaWebConfig serves classpath:/static/ (packaged Angular); SPA fallback to index.html; never hijacks /api/*.
- CorsConfig allows configured local origins for /api/** (dev :4200 and packaged :8080).

6. Algorithms in depth

6.1 Ingest connectivity

OSM lines often cross without sharing digitised vertices. Splitting each polyline into vertex-to-vertex segments and snapping coordinates to a fine coord_key grid (DEDUP_DEGREES ~ 1e-5) merges junctions so the graph is connected for routing.

6.2 Snap complexity

Without a GIST bbox, KNN may scan huge portions of road_edges. With ST_Expand + &&, the planner uses the spatial index. This is essential for pan-India datasets.

6.3 Long-trip hierarchy rationale

Loading every residential street inside a large S-T envelope can exceed timeouts. Hierarchy keeps arterial corridors for the mid-section and retains full connectivity near origin/destination so the path can enter/exit the major network.

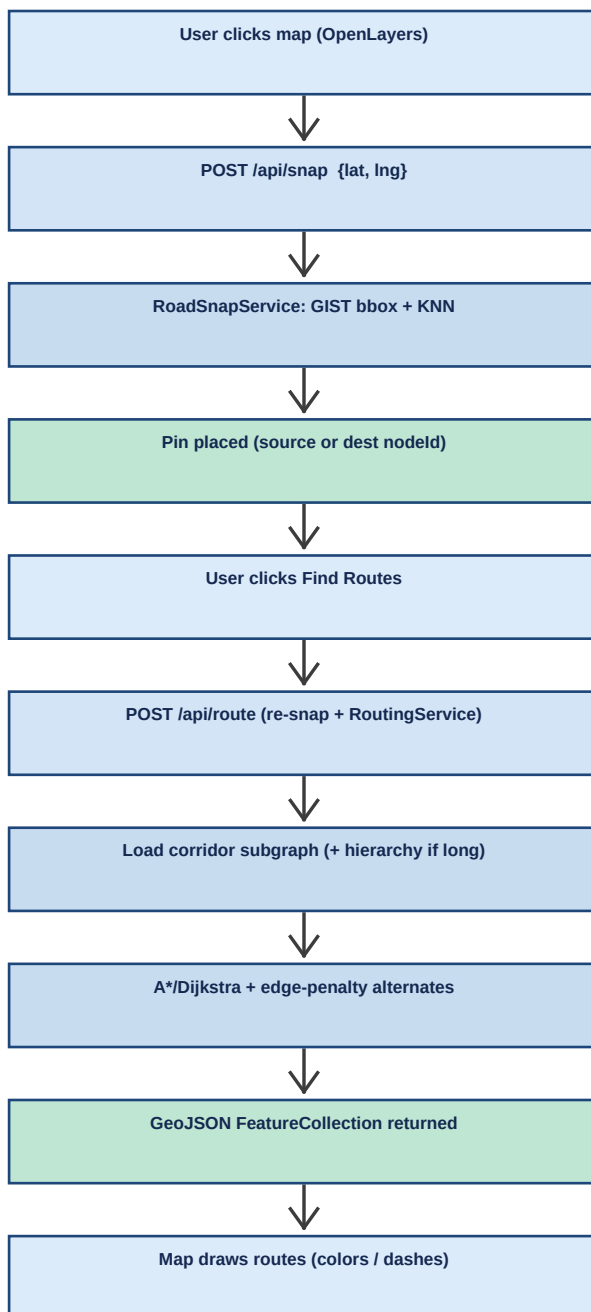
6.4 Basemap: roads_basemap vs broken rasterized.tif

An earlier single-band float GeoTIFF filled entirely with value 255 rendered as near solid yellow (approx RGB 255,255,1) via GeoTools ColorModel + aggressive HTTP caching. rebuild_road_basemap.py burns PostGIS roads into an RGB dark-background TIFF. TileController uses Cache-Control: no-store; frontend tile URL includes ?v=roads1.

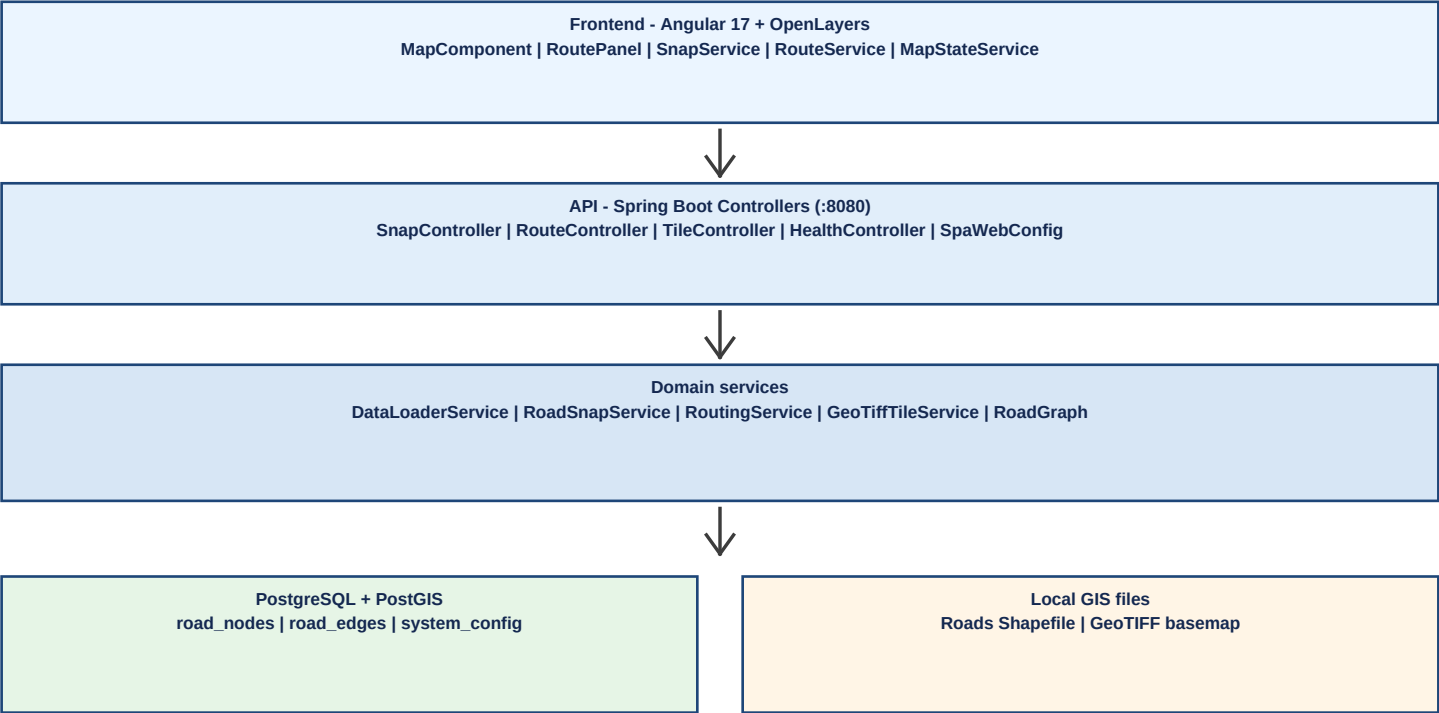
7. Flowcharts and architecture diagrams

7.1 End-to-end interactive workflow

Primary user path: map click to snap, then Find Routes, then GeoJSON overlay on OpenLayers.



7.2 Layered architecture



7.3 Startup / ingest sequence

```
Spring Boot starts -> HTTP :8080 up
  DataLoaderService.scheduleLoad() [background]
    force-reload? TRUNCATE + clear flag
    shp_loaded? skip, mark ready
    else stream SHP -> segments -> upsert nodes -> batch edges
    ANALYZE; shp_loaded=true; DataLoadStatus.ready
  GeoTiffTileService.init() loads geotiff.base.path if present
  GET /api/health reflects dataReady + roadBounds
```

7.4 Routing sequence

```
RouteController: snap source + dest
  RoutingService.route(nodeIds, k, algorithm)
    distM = Haversine(S,T); longTrip = distM >= 12 km
    loadSubgraph(pad, hierarchy=longTrip)
    RoadGraph.kShortestPaths(...)
    if empty: retry all-classes / wider pad
    fetch geometries for used edges
  GeoJson.fromRoutes -> FeatureCollection
```

8. Frontend components and UX flow

Piece	Role
MapComponent	OL map; XYZ tiles; click->snap; pins; draw routes; fit roadBounds
RoutePanelComponent	A*/Dijkstra toggle; K=1..10; Find Routes; Clear; list metrics
ToastComponent	Transient status / errors
MapStateService	RxJS state: pins, routes, algorithm, k, flags, toast
SnapService	POST /api/snap (timeout 60s)
RouteService	POST /api/route (timeout 120s)

UX: first click sets origin (green), second sets destination (orange); Find Routes draws the primary route solid and alternates dashed.

Dev proxy.conf.json forwards /api to :8080. Production packaged mode uses same-origin relative /api URLs.

9. REST API contracts

POST /api/snap

Request: { "lat": number, "lng": number }

Response: { snappedLat, snappedLng, nodeId, nearestRoadName, distanceToRoadM }

Errors: 404 NO_ROAD_NEARBY | 503 DATA_NOT_READY | 504 SNAP_TIMEOUT

POST /api/route

Request: { sourceLat, sourceLng, destLat, destLng,
numAlternatePaths, algorithm }
algorithm = "astar" | "dijkstra" (default astar)

Response: GeoJSON FeatureCollection of LineStrings with
routeIndex, label, totalDistanceMeters, estimatedTimeMinutes

Errors: 404 / 503 / 504 ROUTE_DB_ERROR / 500 ROUTE_FAILED

GET /api/health

status, postgis, dataLoaded, dataReady, dataLoading,
nodeCount, edgeCount, message,
roadBounds { minLng, minLat, maxLng, maxLat }

GET /api/tiles/{z}/{x}/{y}.png

image/png. Cache-Control: no-store, must-revalidate. Transparent outside footprint.

10. Configuration reference

File: georoute-backend/src/main/resources/application.properties (offline-bundle overrides via external application.properties beside the JAR).

Property	Meaning
spring.datasource.*	JDBC URL, user, password (never commit real secrets)
spring.jpa.hibernate.ddl-auto	none - schema from db/init.sql
geotiff.base.path	Basemap GeoTIFF (prefer roads_basemap.tif)
shapefile.base.path	Roads .shp path
shapefile.force-reload	true once to re-ingest, then false
shapefile.node-cache.size	LRU size during ingest (default 300000)
shapefile.attr.*	name / fclass / oneway attribute names
spring.jdbc.template.query-timeout	Global seconds (snap); subgraph uses 90
tiles.cache.dir / enabled	On-disk PNG tile cache
routing.max.speed.kmh	A* heuristic divisor (default 110)
server.port	Default 8080
cors.allowed-origins	Dev :4200 and packaged :8080 origins

11. Dev vs portable offline package

11.1 Development

- setup.bat once: npm install, fonts, mvn deps.
- createdb georoute; psql -f db/init.sql; set DB password in properties.
- mvn spring-boot:run and npm start (UI :4200, API :8080).

11.2 Building offline-bundle (on a connected PC)

- Run package-offline.bat (scripts/package-offline.ps1).
- Builds Angular production; mvn -Poffline-bundle package embeds dist into JAR.
- Assembles offline-bundle/: JAR, application.properties (password PLACEHOLDER), db/init.sql, optional data/*, run-offline.bat/ps1, README.

11.3 Running on an air-gapped PC

- Install JDK 17 + PostgreSQL with PostGIS only (no Node/Maven required).
- createdb + init.sql; edit password in application.properties.
- run-offline.bat -> open http://localhost:8080
- First start may take a long time ingesting roads; later starts skip via shp_loaded.

You can zip and copy the entire offline-bundle folder. Include or exclude heavy GIS files depending on transfer size; paths in application.properties must match.

12. Known pitfalls and performance notes

- JDK: use 17 for GeoTools 29.2; newer JDKs can break the GIS stack.
- Snap timeouts: usually need GIST + ST_Expand prefilter (already implemented).
- Route timeouts on long trips: hierarchy + 90s subgraph timeout + no node-join.
- Yellow map: bad single-band TIFF and/or 7-day browser tile cache (fixed with RGB + no-store + ?v=).
- After changing GeoTIFF: delete tile-cache/ and hard-refresh browser.
- After changing SHP parser: shapefile.force-reload=true once, then false.
- Do not ship real DB passwords in zips; offline packer uses CHANGE_ME.
- Frontend route timeout is 120s to cover long subgraph loads.

13. Summary

GeoRoute cleanly separates UI (Angular/OpenLayers), spatial persistence (PostGIS), and graph search (in-memory RoadGraph on a corridor subgraph). Country-scale performance relies on streaming ingest, bbox-prefiltered snap, hierarchical long-trip loads with a 90s subgraph query budget, and edge-penalty diversification for meaningful alternate routes. Portable deployment collapses UI+API into one fat JAR so air-gapped machines need only Java and PostGIS.

Generated by scripts/generate_tech_doc.py from the implemented codebase. For exact logic, see georoute-backend and georoute-frontend sources.